

Donghyung Lee

DevOps Engineer | Resume & Career Description

E-mail: genius5711@gmail.com Blog(KR): somaz.tistory.com Blog(EN): somaz.blog GitHub: github.com/somaz94

Introduction

Hello, I am Donghyung Lee, a DevOps Engineer.

With a goal of maintaining 99.9% infrastructure availability, I have maximized operational efficiency by establishing processes for codifying all changes (IaC) and establishing systematic incident documentation (postmortem) processes. My greatest strength is the habit of thinking beyond simple operations, contemplating the meaning and purpose of each task, and seeking better directions.

I have experience in designing and building cloud infrastructure on AWS and GCP, and am proficient in service operations in Kubernetes (EKS, GKE, on-premises) environments, IaC implementation using Terraform and Ansible, and CI/CD pipeline construction with GitLab CI/CD, GitHub Actions, ArgoCD, and Jenkins. I also have experience building monitoring and logging systems based on PLG Stack and ELK Stack, and have strengths in operations automation using Python and Bash.

Across 3 companies, I have independently or proactively led cloud migration (AWS → GCP), hybrid architecture design, and on-premises PaaS solution construction, achieving quantitative results including 20% AWS cost reduction through hybrid architecture transition and 50% reduction through GCP migration resource optimization, 50-67% deployment time reduction, and 90%+ manual work automation.

I aim to continue contributing to infrastructure architecture advancement and organizational productivity improvement.

Strengths

Quick Understanding & Adaptation

- Quickly adapt to new environments and tools, contributing immediately to diverse projects

Systematic Documentation & Knowledge Sharing

- Document all work processes and share with team, contributing to organizational knowledge assets

Record-Based Continuous Improvement

- Manage all tasks including incident response, infra changes, and troubleshooting through records to prevent recurrence and drive improvement

Full-Stack Infrastructure Experience

- End-to-end infrastructure experience spanning cloud (AWS, GCP), on-premises, networking, CI/CD, and monitoring

Skills

Cloud	AWS, GCP
Container Orchestration	Kubernetes (EKS/GKE Multi-cluster Ops, On-premises HA with CNI/CSI Optimization)
Infrastructure as Code	Terraform, Ansible, Vagrant, Packer
CI/CD	GitHub Actions, GitLab CI/CD, ArgoCD, Jenkins
Monitoring & Logging	PLG Stack, ELK Stack, EFK Stack, Thanos
Automation	Go, Python, Bash Scripting
Database	MySQL, PostgreSQL, Redis, MongoDB
OS & Servers	Ubuntu, Rocky Linux, CentOS, Debian, GPU Server

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- AWS EKS, GCP GKE production operations & cost optimization (20% reduction via hybrid transition, 50% via GCP migration, savings reinvested in new projects)
- IaC-based infrastructure automation and version control using Terraform (100% GCP resource codification)

- On-premises to cloud hybrid architecture design and migration experience

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- CI/CD pipeline design and enhancement using GitHub Actions, GitLab CI, ArgoCD
- 50-67% deployment time reduction with 3x weekly deployments shortening Time to Market, 95% rollback time decrease (30min → 1-2min)
- Jenkins-based Unity Android/iOS build automation and Slack-integrated deployment notifications

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- Integrated metric/log monitoring with PLG Stack (Prometheus, Loki, Grafana)
- Centralized logging system with ELK Stack (Elasticsearch, Logstash, Kibana, Filebeat)
- 60-70% MTTR improvement, SLI/SLO-based 99.9% service availability achieved

Automation & Efficiency

Operations automation and developer productivity tools using Python and Bash

- 90%+ manual work reduction through data collection, deployment, and document management automation
- Internal LLM service (Ollama + Open WebUI) ensuring 100% internal security guideline compliance and dev team productivity improvement
- Internal tool development: GitLab Webhook doc sync, Slack Bot APK deployment notifications

Professional Experience (Detailed)

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As a dedicated DevOps infrastructure engineer at a game development company, I design, build, and operate AWS-based cloud and on-premises servers. Development and test servers are on-premises, while QA and Production servers are on AWS, operating a hybrid architecture.

Currently providing secondary technical support and operations for the live game 'Souls' in collaboration with the publisher, and building on-premises/AWS infrastructure to improve development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution 100% (Solo design & implementation) 2024.10 ~ Present (In Operation)

Problem

While operating the entire infrastructure on AWS, high cloud costs of \$60,000/month necessitated cost optimization.

Approach

Workload-specific cost analysis revealed that dev/test environments had minimal traffic variation, making cloud elasticity unnecessary.

Designed a hybrid architecture transitioning dev/test to on-premises while keeping QA/Production on AWS.

Unified operational standards through Terraform and Ansible IaC integration (100% IaC coverage) across both environments, achieving consistent infrastructure management without increasing operational complexity.

Troubleshooting

- When configuring ALB Ingress on EKS, needed to route to different backends per game client version. Resolved by combining custom header-based routing rules with Target Groups in ALB conditional routing
- Intermittent timeouts during ClusterIP service communication in Kubernetes IPVS mode. Resolved through Cilium eBPF packet flow analysis and IPVS conntrack threshold tuning, and built a Cilium Network Policy-based zero-trust security model to enhance inter-pod traffic visibility and security
- Pod scheduling delays during EFS mounting. Resolved by optimizing EFS CSI Driver mount options and adjusting StorageClass settings

Results

- Workload analysis-based infra optimization reducing monthly costs by 20% (\$60,000 → \$48,000), reinvesting ~\$144,000 annual savings into new project infrastructure for resource reinvestment cycle
- Offloaded dev servers to on-premises, cutting ~\$1,000/month in AWS resource costs and optimizing resource usage
- Ensured operational stability and prevented failure propagation through environment role separation

Tech Stack

Project 2: On-premises CI/CD Pipeline Construction & Enhancement

Contribution 100% (Full infra & pipeline ownership) 2024.12 ~ Present (In Operation)

Problem

Developers manually built and deployed to servers, taking an average of 30 minutes per deployment with frequent failures due to human errors.

Approach

Built a GitOps-based automated deployment environment combining GitLab CI and ArgoCD. Eliminated Docker-in-Docker dependency using Kaniko for Docker image builds, and configured multi-environment (alpha/review/dev/staging) deployments in a single pipeline.

Troubleshooting

- TLS handshake failure with Harbor registry due to OpenSSL version mismatch in Alpine-based containers. Resolved by replacing base image with OpenSSL 3.x compatible version
- Package installation failure on CI Runner due to GPG key expiration after GitLab upgrade. Automated GPG key renewal process to prevent recurrence
- Build time spikes due to cache invalidation during Kaniko builds. Resolved with `-cache=true --cache-ttl=24h --snapshot-mode=redo` option combination

Results

- 67% deployment time reduction (30min → 10min), 90% manual work automation
- Deployment automation increased weekly deployments 3x+ (1-2 → 5-7), shortening Time to Market for new features, while deployment resource efficiency enabled the dev team to focus on core feature development

Tech Stack

GitLab CI/CD, ArgoCD, Jenkins, Kaniko, Kubernetes, Docker, Harbor

Project 3: Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement)

Contribution 100% (System design & implementation) 2025.06 ~ Present (Operations & Enhancement)

Problem

Server and container logs were distributed across the on-premises environment, making root cause analysis time-consuming during incidents.

Troubleshooting

- Logstash parsing failures due to null bytes in game server logs. Resolved by pre-removing with mutate filter `gsub`
- Elasticsearch field mapping conflicts. Resolved with explicit index template mappings and type conversion filters
- Index read-only due to disk watermark exceeded. Stabilized with ILM policy auto-deleting indices older than 7 days
- [EFK] Fluent Bit NFS WAL write failure. Resolved with custom PVC template patch + upgrade script auto-preserve
- [EFK] Fluentd buffer overflow log loss. Resolved by tuning `chunk_limit_size/flush_interval`
- [ECK] Kibana 9 secure-cookie enforcement blocked HTTP login. Worked around with `tls.disabled + publicBaseUrl http:// + secureCookies:false`
- [ECK] 3.3.x stackconfigpolicy-controller cache-scope mismatch caused reconcile errors every 17 minutes. Worked around with `managedNamespaces:[]` (cluster-wide watch); RBAC was already cluster-scoped
- [ECK] No Fluentd ES9 variant → ensured ES 9.0 compatibility with ES8 variant image via `fluent-plugin-elasticsearch 5.4.4` and `suppress_type_name:true`, verified end-to-end via active marker

Approach (Phase 3 addition)

Phase 3 (ECK Operator + Stack 9.0): With Elastic's Helm Chart updates halted and CRD-based declarative management becoming mainstream, performed an 8.5.1 → 9.0.0 major upgrade and deployed ECK Operator 3.3.2 in the `elastic-system` namespace.

Redefined ES/Kibana as Custom Resources (CR wrapper local chart), executed a zero-downtime cutover after parallel monitoring → logging ns deployment.

Established automatic TLS rotation (1y / 30d), Helm-templated elastic account password, and rolling upgrades with a single-line CR spec.version change.

Results

- 90% log search time reduction, 70% MTTR improvement maintaining 99.9% availability
- 70% memory reduction via EFK migration (Logstash 1GB → Fluent Bit 50MB)
- 1GB daily log real-time processing, unified Helm management with automated upgrades
- ECK migration removed manual ops burden for TLS, passwords, StatefulSet upgrades; Stack 9.0 major upgrade closed a 2-year update gap
- Operator `secureMode` + ServiceMonitor auto-registers controller-runtime metrics into kube-prometheus-stack

Tech Stack

Elasticsearch, Fluent Bit, Fluentd, Logstash, Kibana, Filebeat, ECK Operator, CRD, Custom Resource, Kubernetes, Helm

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server)

Contribution 100% 2025.02 ~ Present

4-1. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Auto-notification to Slack with QR code on Jenkins build completion. 90% QA team delivery time reduction

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.08)

Auto-sync on push via GitLab Webhook + Google Drive API. 80% document management time reduction

4-3. Internal LLM Service Deployment (4 weeks, 2026.01)

Deployed Ollama + Open WebUI on Kubernetes on on-premises GPU server (RTX 3060). Serves 15 developers with avg 100 queries/day, saving ~\$100/month by replacing external AI SaaS subscriptions.

100% internal security guideline compliance and eliminated data leakage risks

4-4. Git Repository Mirroring Tool Development (4 weeks, 2026.02)

Developed Go-based bidirectional Git mirroring tool (git-bridge). Ensured code quality with unit tests and GitHub Actions CI/CD pipeline.

Auto-sync across CodeCommit/GitLab/GitHub for 10 repos with avg 30 sync events/day, 100% automation

4-5. Static File Server - In-house Development (Open Source, 2026.03 ~ Present)

Developed a lightweight Go-based static file server (static-file-server) in-house and operated a Helm repository on GitHub Pages. Features: dark-mode directory listing UI, file preview (image/video/PDF/text), search with URL-hash sharing, multi-select + ZIP batch download, Gzip, Prometheus metrics, JSON logging, automatic APK Content-Type.

Deployed to Kubernetes via an in-house Helm chart (NFS nfs-client-nopath, Retain, 5Gi), fully replacing the legacy halverneus/static-file-server:v1.8.11 (moved to _deprecated). Handles ~30 downloads/day and 5 APK builds/week (daily build). Released as open source under Apache-2.0.

Project 5: Kubernetes Cluster Operations Enhancement

Contribution 100% 2025.12 ~ Present

Enhanced cluster visibility and operational stability by overhauling the monitoring system and automating the K8s upgrade process.

5-1. Monitoring & Alerting Enhancement (2026.01 ~ Present)

18 custom alert rules (Node/Pod/Cilium), 10 custom Grafana dashboards, 4 DB exporters, Ansible node-exporter deployment, Alertmanager Slack severity routing

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ 2026.04)

Kubespray upgrade automation script for pre-flight, backup, compatibility checks + 9-step health check script reducing verification time by 90%.

Upgrade framework across 18 Helm charts + 4 canonical templates synchronizer (check for CI drift detection, apply for batch propagation) maintaining script body consistency.

Automated Kubernetes certificate renewal script + crontab (10 nodes, every 6 months at 3 AM) proactively preventing API Server outages from certificate expiration

Tech Stack

Prometheus, Grafana, Kubespray, Ansible, etcd, Helm

Project 6: Infrastructure Security System

Contribution 100% 2026.04 ~ Present

6-1. Vaultwarden Password Management System (2026.04)

Deployed Vaultwarden on K8s via Helm, GitLab SSO integration, org/collection access control. Automated daily backup (30-day retention) + interactive restore.

Migrated 70 users and 50 secrets into centralized management

Tech Stack

Vaultwarden, Helm, OpenID Connect, GitLab SSO

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game and blockchain startup, leading the large-scale cloud migration project from AWS to GCP.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Reconstruction

Contribution: Infra design/migration lead 70%, CI/CD pipeline 100% 7 months (2023.05 ~ 2023.12)

Problem

Continuously rising AWS costs (\$10,000+/month, high NAT Gateway and data transfer costs), need for GCP global load balancing for multi-region game services, and the legacy Jenkins-based deployment system's configuration complexity and lack of deployment history tracking

Troubleshooting

- Routing issues during AWS ALB → GCP Global LB transition. Reconfigured after analyzing GCP NEG structure
- Data corruption during RDS → Cloud SQL migration due to character set differences. Resolved with pre-validation scripts and step-by-step integrity checks
- Network design issues due to AWS/GCP subnet model differences. Redesigned CIDR and converted to GCP tag-based firewalls

Results

- 50% cloud cost reduction (\$10,000 → \$5,000/month), 50% deployment time reduction, 95% rollback reduction
- Full GCP infrastructure Terraform IaC (TFLint/Checkov static analysis, Terratest unit testing), 100% deployment history tracking via Git commits
- Minimized downtime with resource-copy migration; final DNS cutover executed during an approximately 2-hour scheduled maintenance window aligned with game service operations

Tech Stack

GCP (GKE, Cloud SQL, Cloud Storage, VPC, Global LB), Terraform, GitLab CI, ArgoCD, GitHub Actions, Helm

Project 2: Game Data Collection Automation System

Contribution 100% 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, costing analysts 2-3 hours daily, with frequent data loss caused by human error

Results

- 95% data collection automation, 0% data gap rate, 10x throughput increase

Tech Stack

Python, Cloud Functions, Cloud Scheduler, Google Sheets API

Project 3: PLG Stack Monitoring System

Contribution 100% 1 month (2024.04)

Problem

No real-time monitoring system for Kubernetes clusters and applications, allowing only reactive responses with excessive time spent on root-cause analysis

Results

- SLI/SLO establishment and Alertmanager alarm noise optimization, 80% proactive detection, 60% response time reduction, availability 99.5% → 99.9%

Tech Stack

Prometheus, Loki, Grafana, Promtail, Fluent-bit, Slack API

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients. Designed on-premises cloud infrastructure for financial and public sector clients, conducted operations training.

Key Projects: PaaS Solution · DP Training · Certificate Automation · Ansible Configuration Management

Results

- PaaS solution for 5 clients, Kubernetes HA 99.9% availability
- Established DP's autonomous Kubernetes operations (6-month training, avg 10 attendees · 4 hrs per session)
- 10x certificate renewal extension (1yr → 10yr), 50hrs annual ops savings
- Ansible-based server provisioning and configuration management automation eliminating configuration drift issues

Tech Stack

Kubernetes, OpenStack, Ceph, Ansible, kubeadm, Rocky Linux/CentOS

ETech System

2022.01 ~ 2022.04

System Engineer

Hardware Server and SAN storage construction. Transitioned to DevOps field for career growth.

Opensource Projects

Kubernetes CRD

[k8s-namespace-sync](#) — Automatically sync Kubernetes namespace resources across clusters

[helios-lb](#) — Custom load balancer controller for Kubernetes

[network-policy-generator](#) — Kubernetes NetworkPolicy auto-generation controller

GitHub Actions

[Compress Decompress Action](#) — Compress/decompress files in workflows

[Image Tag Updater](#) — Auto-update image tags in YAML/Helm files

[Go Git Commit Action](#) — Go-based Git commit creation and push

[Extract Commit Action](#) — Extract Git commit SHA, message, and author information

[Multi Git Mirror Action](#) — Automate mirroring across multiple Git repositories

[Kube Diff Action](#) — Compare and report Kubernetes resource changes

Ansible Galaxy

[Ansible K8s IAC Tool](#) — Automated K8s and IaC tool installation collection

[Ansible User Management](#) — Linux user and SSH key management role

DevOps Tools

[bash-pilot](#) — AI-powered Bash command assistant CLI tool

[git-bridge](#) — Git repository mirroring and sync CLI tool

[static-file-server](#) — Go-based static file server with directory UI, file preview, search, ZIP batch download

Chrome Extensions

[Dev Toolkit](#) — Developer utility collection

[QRify](#) — Instant QR code generation for URLs/text

Education

Chungnam National University - Public Administration (Transfer)	2018.03 ~ 2020.08
National Institute for Lifelong Education - Business Administration (Academic Credit Bank)	2016.09 ~ 2018.02

Certifications

Engineer Information Processing	2021.11
AWS Certified Solution Architect - Associate (Expired)	2021.11
Network Administrator Level 2	2021.10
Linux Master Level 2	2021.10
Computer Specialist Level 1	2021.04
TOEIC Speaking Test - 130/Intermediate Mid 3 (Expired)	2021.02

Study

T101(Terraform) Study	2023.08 ~ 2023.10
AEWS(EKS) Study	2023.04 ~ 2023.06
PKOS(Kubernetes) Study	2023.01 ~ 2023.02
Cloud Security Training	2022.02 ~ 2022.04
Security Solution Operation Specialist Training Program	2021.07 ~ 2021.12